

CS599 期末大作业报告

企业级应用软件设计与开发（AI 驱动的软件开发与 Agentic AI）

课程名称	企业级应用软件设计与开发
项目名称	ResearchPilot: Agentic AI Research Assistant
方向	方向一：Agentic AI 原生开发
学号	2025303065
姓名	刘迪
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日
部署地址	http://60.205.203.170:3000

本报告根据课程要求整理项目背景、SDD 规格、系统架构、关键实现、测试评估、扩展计划与课程总结。

目录

目录	2
一、选题背景与设计思想	3
二、Specs 规格文档	3
2.1 Product Spec 摘要	4
2.2 Architecture Spec 摘要	4
2.3 API Spec 摘要	4
三、系统架构与设计	5
3.1 三层架构图	5
3.2 Agent 交互流程	5
3.3 数据流设计	6
3.4 状态模型	6
四、关键实现与代码展示	6
4.1 Agent 核心循环	6
4.2 工具定义与 MCP Server	7
4.3 LLM 接入与安全配置	7
4.4 前端实现	7
4.5 AI IDE 使用记录	7
五、测试与评估	8
5.1 功能测试	8
5.2 自动化测试与构建验证	8
5.3 Agent 行为评估	9
5.4 Demo 过程	9
六、系统升级与扩展	12
七、课程总结	12

一、选题背景与设计思想

研究生课程学习、论文开题、技术选型和工程调研都需要快速完成文献检索与综述写作。传统调研流程通常依赖人工在多个学术平台中检索论文，再逐篇阅读摘要、整理方法、归纳趋势并撰写报告。这一流程存在四类典型问题：

- 检索与筛选成本高：同一个主题需要在多个平台反复搜索，关键词调整依赖人工经验。
- 证据链分散：论文标题、摘要、链接、年份、作者和阅读笔记常分散在浏览器、笔记软件和本地文件中。
- 报告质量不稳定：不同时间、不同资料来源会导致综述结构、引用完整性和事实一致性波动。
- 过程不可观测：最终报告往往只展示结论，缺少“Agent 做了什么、依据是什么、质量如何”的可追踪过程。

ResearchPilot 将学术调研抽象为一个 Agentic AI 原生任务：用户输入研究主题后，系统自动规划研究问题、检索论文、总结论文、构建任务级知识库、生成综述报告并评估报告质量。项目重点不是做一个普通聊天机器人，而是把“调研”拆解为多个职责清晰、状态可追踪、结果可评估的 Agent 节点。

本项目的设计思想包括：

- 规格驱动开发：先定义 Product Spec、Architecture Spec 和 API Spec，再实现前后端与 Agent 工作流。
- 多 Agent 协作：Planner、Search、Reader、RAG Store、Synthesis、Writer、Evaluator 分工处理调研闭环。
- 工具调用与 MCP：将论文检索、文档存储、检索问答、报告生成和质量评估封装为工具能力，并提供 MCP Server。
- 任务记忆与 RAG：使用 SQLite 保存任务状态、论文、日志和评估结果；使用 Chroma 或 JSON 索引保存任务级知识库。
- 可观测执行轨迹：前端以时间线和日志方式展示 Agent 执行过程，失败原因直接暴露在任务详情中。
- 质量评估闭环：Evaluator Agent 从相关性、引用完整性、结构清晰度、事实一致性四个维度评价报告。

技术路线如下：

用户主题	Planner	Search	Reader/RAG	Writer	Evaluator
研究意图	关键词/问题/大纲	论文检索	论文总结与知识库	Markdown 报告	质量评分

二、Specs 规格文档

本项目采用 SDD（规格驱动开发）方法，将需求、架构和接口先文档化，再进入实现。仓库中的规格文档包括：

规格文档	文件路径	核心内容
Product Spec	docs/product_spec.md	项目背景、用户目标、MVP 范围、非目标、成功指标和风险处理

规格文档	文件路径	核心内容
Architecture Spec	docs/architecture_spec.md	分层架构、Agent 流程、状态模型、可扩展点
API Spec	docs/api_spec.md	任务创建、任务查询、报告获取、报告下载、知识库问答接口
Test Report	docs/test_report.md	测试目标、单元测试、集成流程、真实运行前置条件

2.1 Product Spec 摘要

Product Spec 将用户目标定义为“输入研究主题后自动获得结构化研究报告，并能查看 Agent 轨迹、论文来源、报告和评估结果”。MVP 范围包括任务创建、Planner 规划、Search 检索、Reader 总结、RAG 写入、Writer 写作、Evaluator 评分以及 Next.js 前端展示。

非目标包括用户登录、复杂权限管理、PDF 全文解析、多用户协作和复杂爬虫。该边界保证项目在课程周期内优先完成 Agent 闭环，而不是扩散到通用论文管理系统。

2.2 Architecture Spec 摘要

Architecture Spec 将系统拆成四层：

- 前端展示层：Next.js + Tailwind CSS，用于任务创建、执行轨迹、论文卡片、报告预览、质量评分和知识库问答。
- 业务接口层：FastAPI 提供 REST API，负责任务生命周期、报告下载和问答入口。
- 智能体执行层：ResearchWorkflow 负责编排 Planner、Search、Reader、RAG Store、Synthesis、Writer、Evaluator。
- 数据与工具层：SQLite、Chroma/JSON、OpenAlex、arXiv、MCP Server 和 Markdown 报告文件。

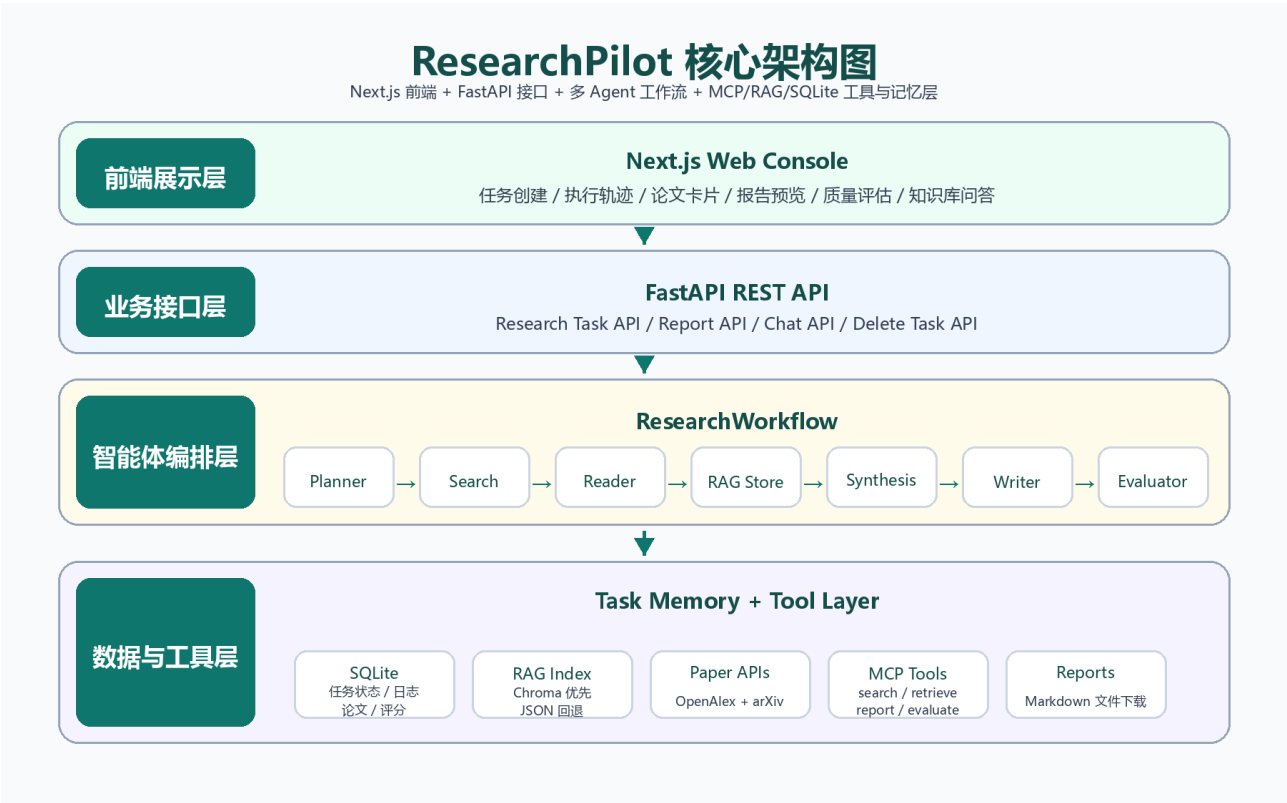
2.3 API Spec 摘要

核心 API 包括：

方法	路径	用途
POST	/api/research-tasks	创建研究任务并启动 Agent 工作流
GET	/api/research-tasks	查询最近任务
GET	/api/research-tasks/{task_id}	查询任务详情、日志、论文和评估
DELETE	/api/research-tasks/{task_id}	删除任务及其关联数据
GET	/api/research-tasks/{task_id}/report	获取 Markdown 报告内容
GET	/api/research-tasks/{task_id}/report/download	下载报告文件
POST	/api/chat	基于任务知识库继续问答

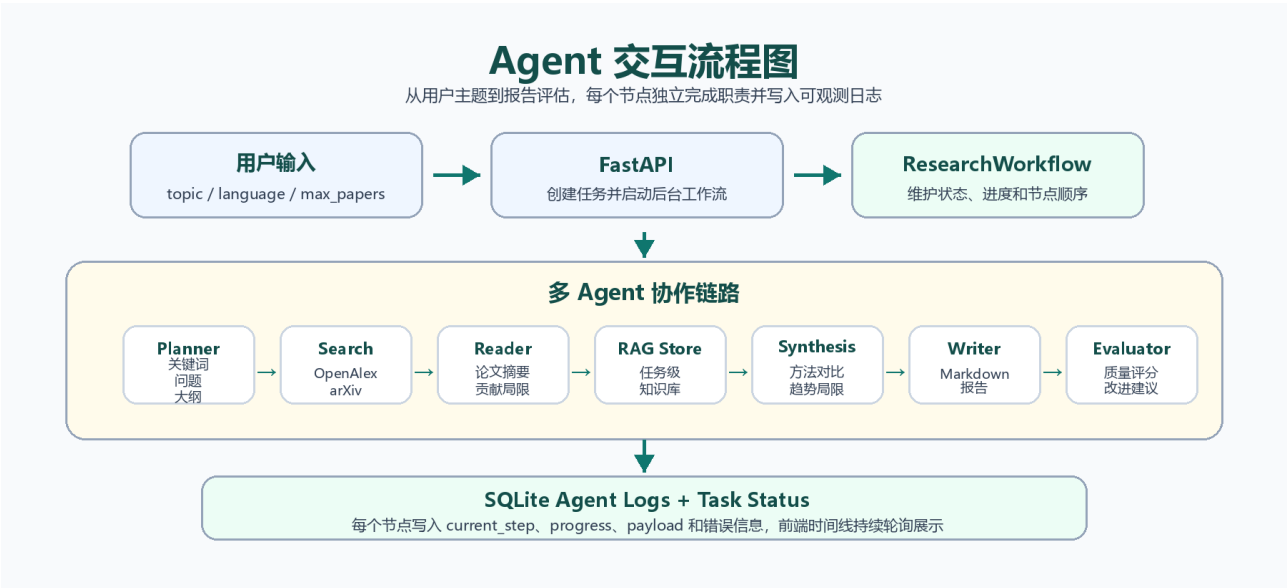
三、系统架构与设计

3.1 三层架构图



系统采用前后端分离架构。前端负责展示和交互，后端负责 API、任务状态和 Agent 工作流，数据层负责任务记忆、论文元数据、报告文件与知识库索引。这样可以在不改变前端页面的情况下替换模型、论文源或向量库，也可以在后续加入 SSE 实时推送、任务队列和多用户权限。

3.2 Agent 交互流程



Agent 按“规划、检索、阅读、存储、综合、写作、评估”的顺序协作，每一步都会写入任务状态和执行日志。

3.3 数据流设计



数据从用户主题进入任务记忆，再经过外部论文源、RAG 索引、报告文件和评估结果，最终回到前端展示；删除任务时会同步清理全部关联数据。

3.4 状态模型

任务状态从 `created` 进入 `running`，并依次流转到 `planner`、`search`、`reader`、`rag_store`、`synthesis`、`writer`、`evaluator` 和 `completed`。任一节点出现异常时，任务进入 `failed`，错误写入日志并显示在前端任务详情页。

四、关键实现与代码展示

4.1 Agent 核心循环

核心工作流位于 `src/api/app/agents/workflow.py`。该文件的 `ResearchWorkflow.run()` 将一个研究任务串联成完整闭环：

```
self.db.update_task(task_id, status="running", current_step="planner", progress=5)
plan = self._planner(task_id, task["topic"], task["language"])

self.db.update_task(task_id, current_step="search", progress=20)
papers = self._search(task_id, task["topic"], plan["keywords"], task["max_papers"])

self.db.update_task(task_id, current_step="reader", progress=42)
papers = self._reader(task_id, task["topic"], task["language"], papers)
self.db.replace_papers(task_id, papers)

self.db.update_task(task_id, current_step="rag_store", progress=58)
self._rag_store(task_id, papers)

self.db.update_task(task_id, current_step="synthesis", progress=70)
synthesis = self._synthesis(task_id, task["topic"], task["language"], papers)

self.db.update_task(task_id, current_step="writer", progress=82)
report_path, report = self._writer(task_id, task, plan, papers, synthesis)

self.db.update_task(task_id, current_step="evaluator", progress=92)
evaluation = self._evaluator(task_id, task["topic"], task["language"], report, papers)
```

该实现体现了多步骤状态管理：每个节点执行前都会更新 `current_step` 和 `progress`，每个节点完成后都会写入 Agent 日志，前端可以持续轮询并展示进度。

4.2 工具定义与 MCP Server

工具层位于 `src/api/app/mcp_server/server.py`。系统将论文检索、详情获取、知识库存储、知识库检索、报告写作和报告评估暴露为 MCP 工具：

```
@mcp.tool()
def search_papers_tool(topic: str, keywords: list[str] | None = None, max_papers: int = 8) -> list[dict]:
    return search_papers(topic, keywords=keywords, max_papers=max_papers)

@mcp.tool()
def retrieve_documents_tool(task_id: str, query: str, top_k: int = 4) -> list[dict]:
    return vector_store.retrieve_documents(task_id, query, top_k=top_k)

@mcp.tool()
def evaluate_report_tool(topic: str, report: str, papers: list[dict]) -> dict:
    return evaluate_report(topic, report, papers)
```

这部分满足课程中对工具使用、Function Calling 和 MCP 协议的要求。即使前端调用的是 REST API，后端仍然具备可复用的工具层。

4.3 LLM 接入与安全配置

LLM 接入层位于 `src/api/app/llm/client.py`，支持 DeepSeek 和 OpenAI-compatible Provider。真实 API Key 不写入代码，而是通过 `.env` 配置：

```
LLM_PROVIDER=openai_compatible
OPENAI_COMPATIBLE_BASE_URL=https://token.sensenova.cn/v1
OPENAI_COMPATIBLE_API_KEY=your_api_key_here
OPENAI_COMPATIBLE_MODEL=sensenova-6.7-flash-lite
```

LLM 客户端对空响应做了重试处理，并在 Planner、Evaluator 等结构化输出场景中增加 JSON 解析容错，降低模型输出格式不稳定导致的失败概率。

4.4 前端实现

前端位于 `src/web`，使用 Next.js App Router。关键组件包括：

组件	职责
TopicInputCard	创建研究任务
TaskStatusTimeline	展示 Agent 执行步骤
PaperCard	展示论文标题、作者、年份、摘要、相关性和 Agent 总结
ReportViewer	渲染 Markdown 报告并提供下载入口
EvaluationPanel	展示质量评分与建议
ChatPanel	基于任务知识库继续问答

首页最近任务列表支持删除任务。删除操作会调用 `DELETE /api/research-tasks/{task_id}`，后端同步清理 SQLite、报告文件和 RAG 索引。

4.5 AI IDE 使用记录

开发过程中使用 Trae CN / Codex 辅助完成需求拆解、接口设计、错误日志定位、代码生成、测试补充和报告整理。典型协作场景包括：

- 根据课程要求整理 ResearchPilot 开发计划。

- 调试 OpenAI-compatible API 接入和模型 JSON 输出问题。
- 增强 Planner / Synthesis 对非严格 JSON 输出的容错能力。
- 增加最近任务删除功能，并补充数据库和 RAG 索引清理测试。
- 生成 README 和课程报告。

五、测试与评估

5.1 功能测试

本项目按“任务创建 -> Agent 执行 -> 论文结果 -> 报告生成 -> 质量评估 -> 任务清理”的主链路进行功能测试。测试环境为本地 Windows 开发环境，后端运行 FastAPI，前端运行 Next.js，LLM Provider 使用 OpenAI-compatible API，论文检索源使用 OpenAlex / arXiv。

测试项	操作步骤	预期结果	实际结果
任务创建	在首页输入主题“fastlivo2在资源受限平台上的应用”，选择中文，最大论文数为20，点击开始研究	后端创建 research task ，前端跳转任务详情页	通过，任务进入执行页面，并在最近任务中显示完成记录
Agent 状态流转	观察 Planner、Search、Reader、RAG Store、Synthesis、Writer、Evaluator 节点	所有节点按顺序完成，进度达到 100%	通过，状态为 completed ，日志显示真实 LLM Provider、检索 20 篇论文、总结 20 篇论文
论文检索结果	切换到论文结果页	展示论文来源、年份、相关性、摘要、Agent 评价和原文链接	通过，页面展示 OpenAlex 论文卡片，相关性评分最高为 98%
报告预览与下载	切换到研究报告页	Markdown 报告可阅读、可复制、可下载	通过，报告按研究背景、代表性论文、方法分析等章节渲染
报告质量评估	切换到质量评估页	展示相关性、引用完整性、结构清晰度、事实一致性和综合评分	通过，综合评分 9.0，并给出多条改进建议
删除任务	在首页最近任务列表点击删除按钮	清理任务、论文、日志、评估、报告文件和 RAG 索引	通过，任务从最近任务列表移除，后端删除关联数据
异常可观测性	人为触发 API Key、论文源或 LLM 输出异常	任务进入 failed ，错误写入 Agent 日志	通过，失败原因会在任务详情页日志区域显示

功能测试表明，系统已经完成从主题输入到报告交付的核心业务闭环，并且能够展示 Agentic AI 系统区别于普通问答系统的执行过程。

5.2 自动化测试与构建验证

当前后端自动化测试位于 `src/api/tests`，覆盖 workflow、LLM 客户端、数据库清理和 RAG 索引清理四类关键行为：

测试文件	验证内容
test_workflow.py	使用 Fake LLM 和 stub 论文检索验证完整 Agent 闭环
test_llm_client.py	验证 LLM 客户端对空响应进行重试
test_database.py	验证删除任务时清理论文、日志、评估等关联记录
test_vector_store.py	验证删除任务时清理本地 RAG 索引

后端测试命令：

```
cd src/api
pytest
```

最近一次验证结果：

```
4 passed in 2.42s
```

前端构建命令：

```
cd src/web
npm run build
```

最近一次构建结果为编译成功，静态页面包括首页、知识库问答页和任务详情页。

5.3 Agent 行为评估

Evaluator Agent 从四个维度给报告打分：

相关性	引用完整性	结构清晰度	事实一致性	综合评分
是否围绕主题	是否保留论文来源	章节是否合理	是否受证据约束	0-10 分汇总

该评估不是简单判断“是否生成文本”，而是对报告是否围绕用户主题、是否保留论文来源、章节是否清晰、是否避免明显脱离证据的内容进行量化。评分结果会持久化到 SQLite，并显示在前端评分面板中。

5.4 Demo 过程

以下截图来自项目实际运行结果，演示主题为“fastlivo2在资源受限平台上的应用”。

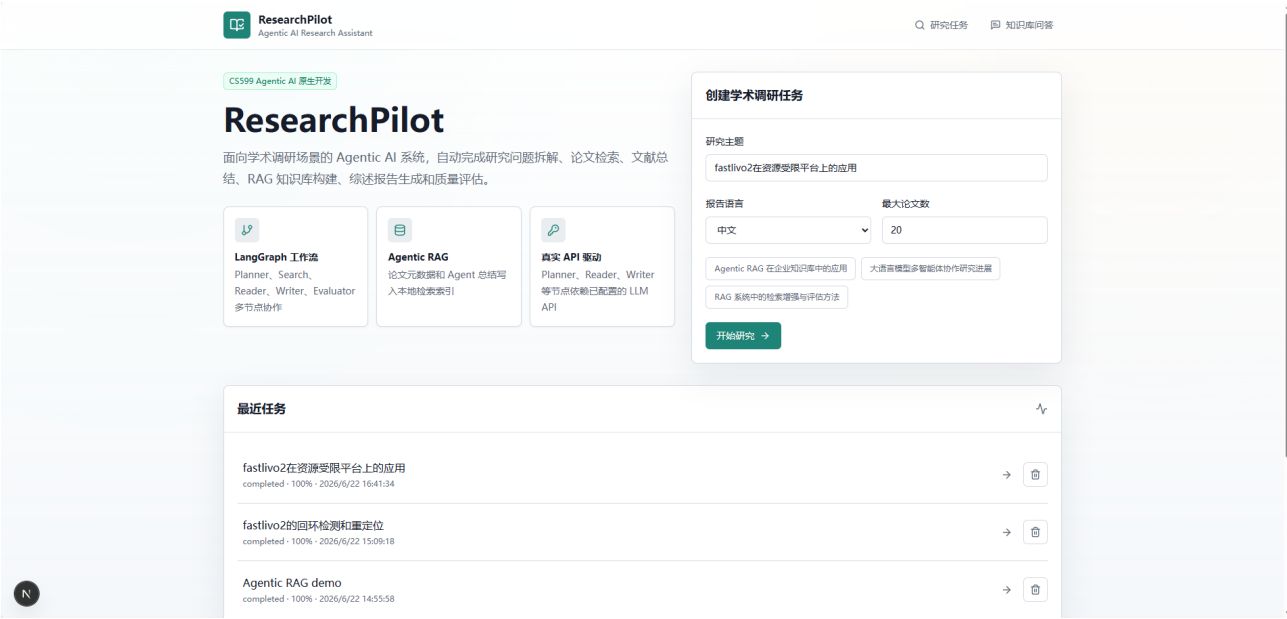


图 5-1 首页任务创建与最近任务列表。用户可以输入研究主题、选择报告语言和最大论文数量，系统完成后会在最近任务列表中显示任务状态，并支持删除历史任务。

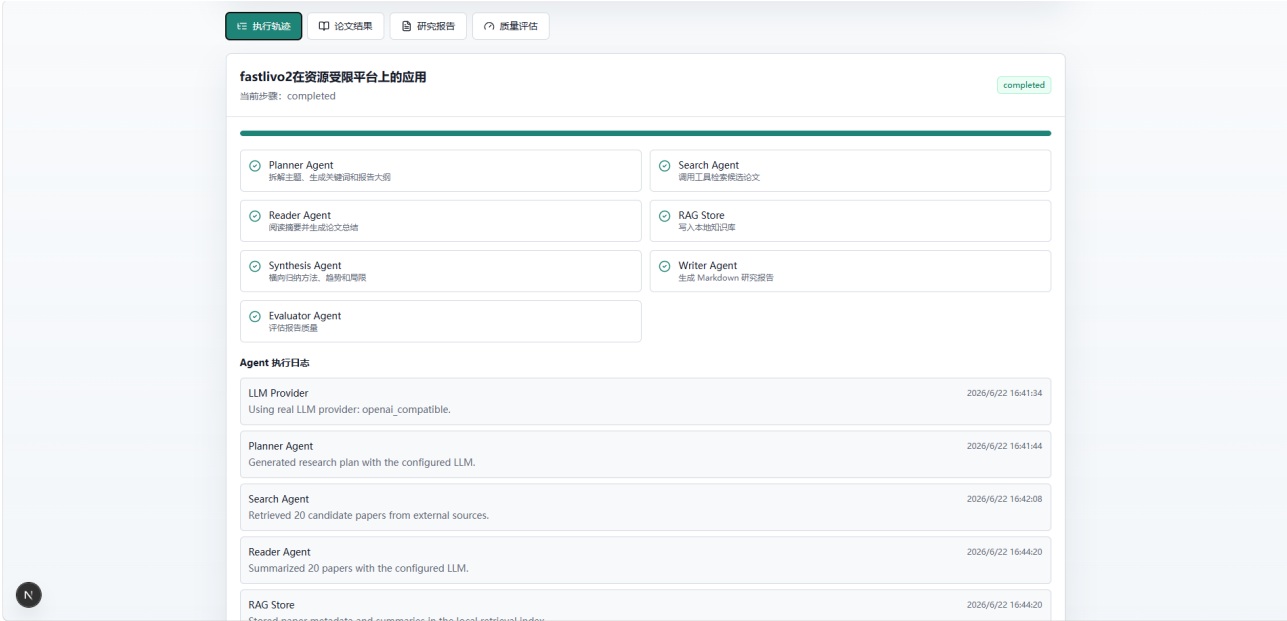


图 5-2 Agent 执行轨迹。任务状态为 completed，Planner、Search、Reader、RAG Store、Synthesis、Writer、Evaluator 均已完成，日志展示真实 LLM Provider、论文检索数量和论文总结数量。

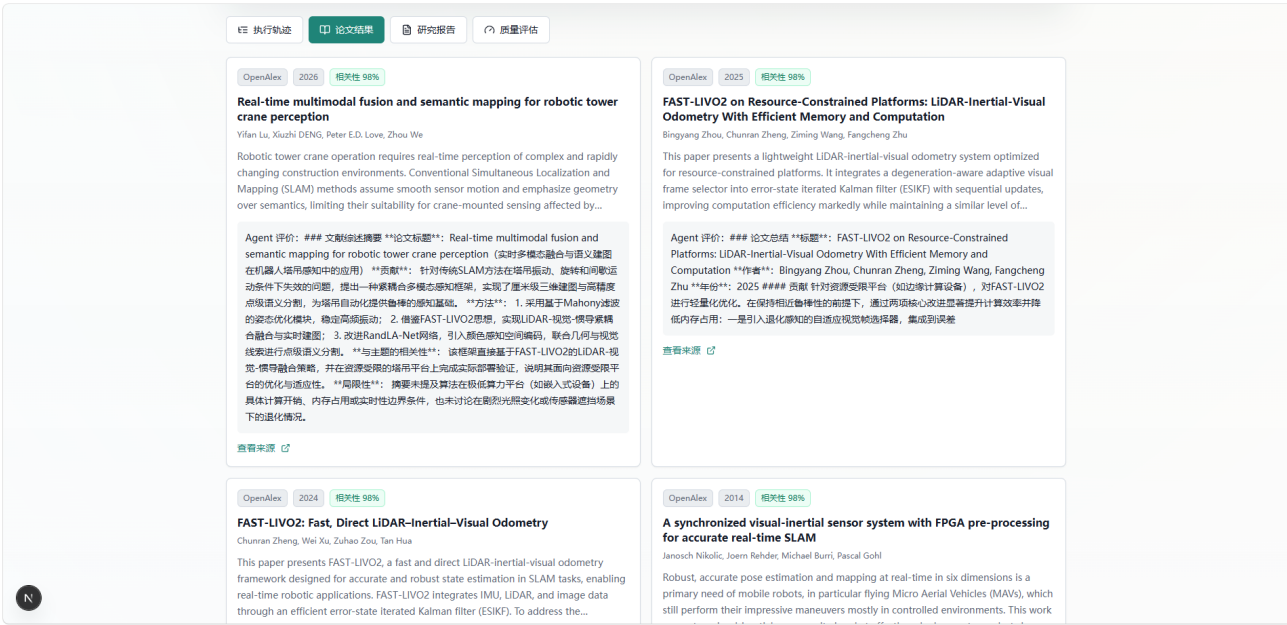


图 5-3 论文检索结果。页面以卡片形式展示 OpenAlex 论文来源、年份、相关性评分、摘要、Agent 评价和原文链接，证明 Search 与 Reader 节点已经形成可追踪证据。

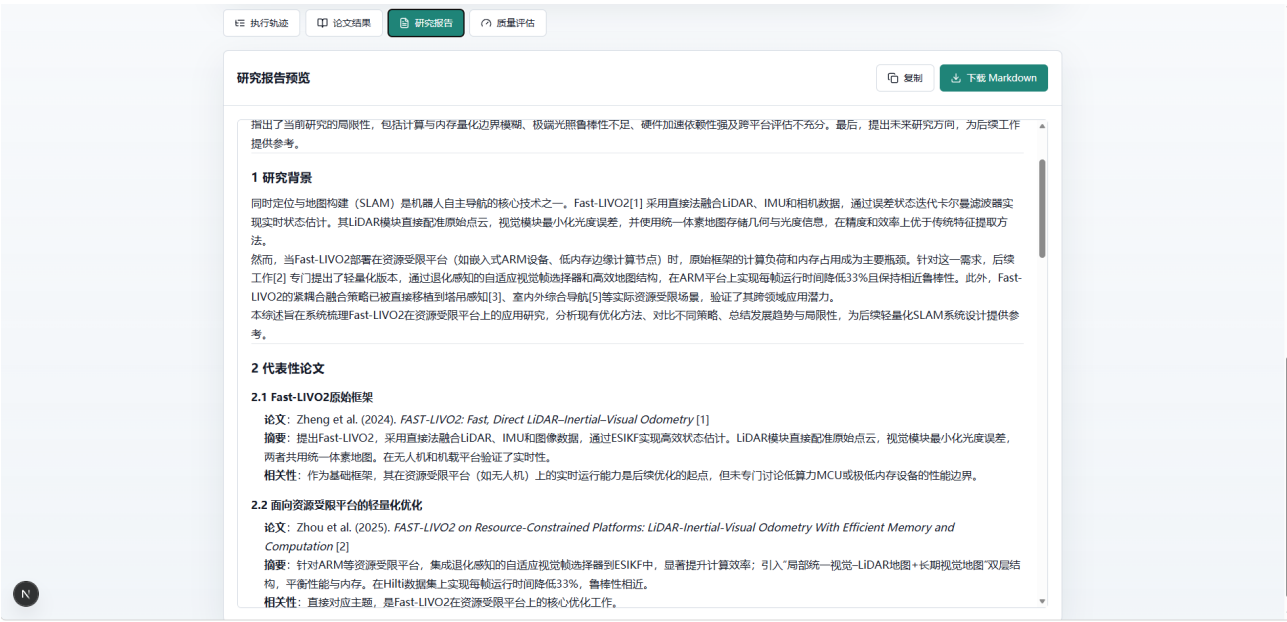


图 5-4 研究报告预览。Writer Agent 生成 Markdown 综述报告，前端支持滚动预览、复制和下载 Markdown，报告内容按研究背景、代表性论文和方法分析组织。



图 5-5 报告质量评估。Evaluator Agent 输出相关性、引用完整性、结构清晰度、事实一致性和综合评分，并给出后续改进建议。

六、系统升级与扩展

ResearchPilot 当前版本完成了 Agentic AI 学术调研的核心闭环。下一阶段可以从以下方向升级：

- workflows 升级：将当前显式顺序 workflow 迁移为 LangGraph StateGraph，增加条件路由、失败重试和人工确认节点。
- 检索源扩展：增加 Semantic Scholar、Crossref、IEEE、ACM 等论文源适配器。
- 全文阅读：接入 PDF 下载与解析，让 Reader Agent 读取全文方法、实验和结论，而不仅依赖摘要。
- 评估增强：引入 RAGAS、DeepEval 或自定义 benchmark，对事实一致性和引用覆盖率做更细粒度评估。
- 实时可观测性：使用 SSE 或 WebSocket 替代轮询，实时推送 Agent 日志。
- 任务队列：使用 Celery / RQ / Dramatiq 处理长任务，提高并发和稳定性。
- 多用户与权限：加入用户登录、任务归属、共享和协作评审。
- 部署升级：部署到云服务器，提供公网可访问 URL，并增加 HTTPS、日志采集和异常告警。

七、课程总结

本项目最大的收获是从“写一段功能代码”转向“编排一个可观测的智能体流程”。在普通 Web 系统中，后端通常只接收请求并返回结果；而 Agentic AI 系统需要考虑任务状态、工具调用、模型不稳定输出、外部 API 错误、知识库记忆、质量评估和用户可解释性。

通过 ResearchPilot 的实现，我对以下工程能力有了更具体的认识：

- SDD 能降低需求漂移。先写规格文档可以明确 MVP 边界，避免在有限时间内扩散到登录、权限、复杂爬虫等非核心功能。
- Agent 节点需要清晰职责。Planner、Search、Reader、Synthesis、Writer、Evaluator 分工后，系统更容易调试和展示。
- 模型输出必须做工程化约束。实际接入 OpenAI-compatible API 后，模型可能返回空响应、非 JSON 或不完整结构，必须有重试、解析和兜底策略。
- 可观测性是 Agent 系统的核心体验。前端时间线和日志让答辩时可以清楚说明系统不是普通 RAG，而是多步骤 Agent 工作流。
- 安全配置必须前置。API Key 只能放在 .env，不能进入代码、README、报告或 GitHub 提交历史。

对课程的建议是：可以增加一次围绕“Agent 评估与可观测性”的专题实践，因为真实 Agent 项目的难点不仅在调用模型，还在于如何判断每一步是否可靠、如何复现错误、如何设计可解释的评估指标。

总体来看，ResearchPilot 已经完成课程要求中的方向一核心目标：从零构建具备实际价值的 Agentic AI 系统，并覆盖 SDD、MCP / Tool Use、记忆机制、状态管理、多 Agent 协作、RAG 和可观测评估等关键技术要素。